

프로젝트 명세서

영상처리 딥러닝을 활용한 인공지능 무인운반차량
제작
(임베디드 트랙)

“로그 데이터 관리 PJT”

2 회차

목차

1. 로그 데이터 관리 PJT	3
1.1 목표	3
1.2 준비사항	3
1.3 작업 순서	5
1.4 요구사항	5
1.5 참고자료	9
1.6 프로젝트 결과	9

1. 로그 데이터 관리 PJT

1.1 목표

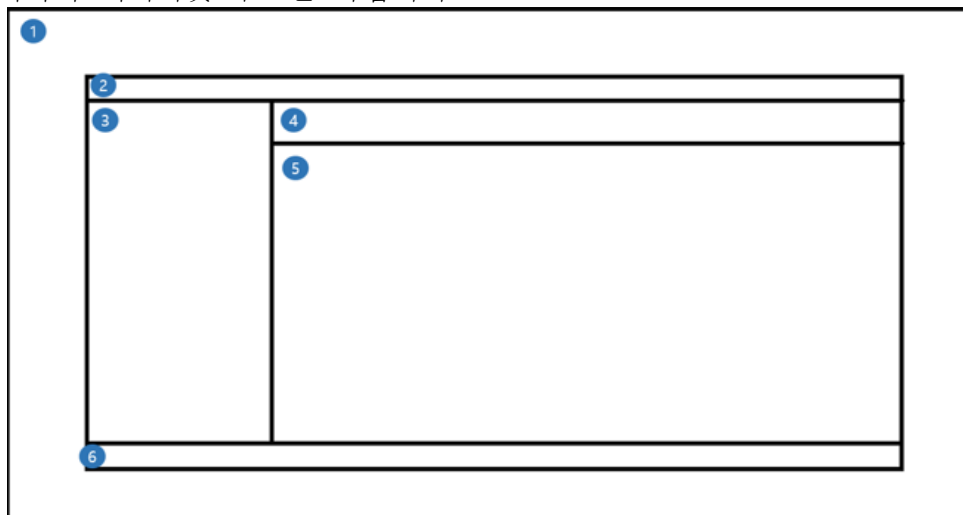
이번 ‘Firebase Firestore 및 Functions 를 활용한 데이터 로그 시스템 구축’ 프로젝트의 목표는 다음과 같습니다.

- Firebase Firestore 를 활용하여 시스템에서 발생하는 원시 로그를 수집하고 저장하는 데이터 관리 능력을 향상할 수 있습니다.
- Firebase Functions 를 사용하여 서버리스 환경에서 데이터를 실시간으로 가공하고 정제하는 백엔드 로직 구현 능력을 배양할 수 있습니다.
- 정제된 데이터를 사용자가 효율적으로 조회할 수 있도록 필터링 및 검색 기능을 포함한 데이터 시각화 인터페이스를 설계할 수 있습니다.

1.2 준비사항

1) 프로젝트 구조

- 이번 프로젝트는 기존 인포테인먼트 시스템의 Setting 버튼을 통해 진입하며, 수집된 차량 데이터를 가공하여 실시간으로 출력하는 데이터 파이프라인 및 시각화 레이아웃 구조를 가집니다.



<그림 1> 로그 시각화 애플리케이션 인터페이스 레이아웃 설계도

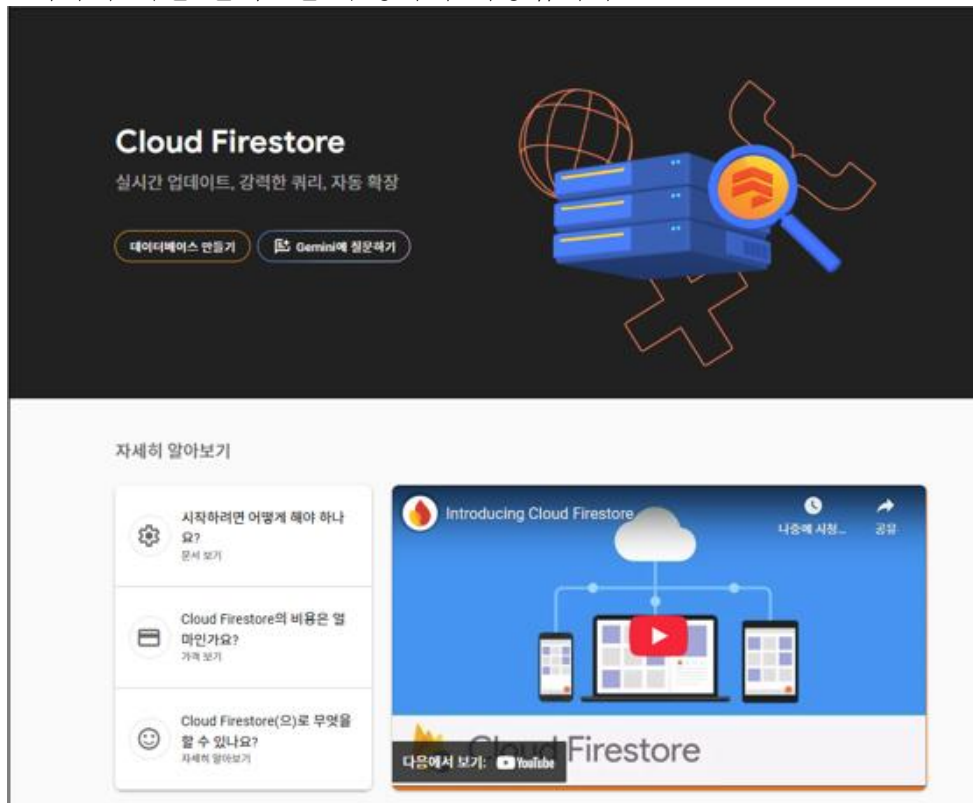
■ 로그 시각화 화면 UI

- ✓ ① 배경화면 영역: 자동차 대시보드 환경을 묘사하는 최하단 배경 계층입니다.

- ✓ ② 태블릿 Header 영역: 현재 시각, 네트워크 상태, 사용자 프로필 등 시스템 정보를 표시합니다.
- ✓ ③ 설정 및 메뉴 영역: 로그 관리 시스템의 주요 설정이나 다른 설정 카테고리 이동을 위한 사이드바 영역입니다.
- ✓ ④ 로그 필터링 조건 영역: 시작/종료 날짜, 로그 레벨(INFO, ERROR 등), 키워드 검색을 위한 입력 필드를 배치합니다.
- ✓ ⑤ 로그 시각화 메인 영역: 정제된 로그 데이터가 리스트 형태로 출력되며, 실시간 업데이트를 통해 최신 정보를 노출합니다.
- ✓ ⑥ 태블릿 Footer 영역: 홈 복귀 버튼 및 주요 메뉴 바로가기가 고정된 하단 제어 바 영역입니다.

1) 사용 데이터

- 실시간 업데이트와 강력한 쿼리 기능을 제공하는 NoSQL 기반의 Cloud Firestore를 메인 데이터베이스로 활용합니다.
- 시스템에서 발생하는 원시 로그(Raw Log)와 정제된 텔레메틱스 데이터를 관리하기 위한 컬렉션을 구성하여 사용합니다.



<그림 2> Cloud Firestore 데이터베이스 관리 화면

2) 개발언어 및 툴

- 데이터 정제 로직 및 동적인 시각화 화면 구현을 위해 HTML5, CSS3, JavaScript를 기본 개발 언어로 사용합니다.

- 효율적인 코드 작성 및 디버깅을 위한 IDE 로 Visual Studio Code 를 활용합니다.
- Firebase 프로젝트 설정 및 Functions 배포를 위해 Firebase CLI 환경을 구축하여 사용합니다.

3) 필수 라이브러리 / 오픈소스

- 서버측 로직 구현 및 데이터 정제 파이프라인 구축을 위해 Firebase Firestore 와 Firebase Functions 라이브러리를 사용합니다.
- 외부 데이터 통신 및 HTTP 요청 처리를 위해 Axios 라이브러리를 활용합니다.
- 별도의 서버 구축 없이 Firebase 에서 제공하는 서버리스 환경과 클라이언트 연동 라이브러리를 적용합니다.

1.3 작업 순서

- 1) 페어와 같이 요구사항(필수/심화)을 확인합니다.
- 2) 개발 사항 및 수정(개선) 등 할 일을 정하고 역할을 분담합니다.
- 3) 기존에 설계된 관통 프로젝트의 구조를 확인하여 데이터가 연동될 위치를 파악합니다.
- 4) Firebase Functions 를 사용하여 원시 로그를 정제하고 전용 저장소에 저장하는 데이터 파이프라인을 구현합니다.
- 5) 정제된 로그 데이터를 기반으로 필터링 및 검색 기능이 포함된 시각화 화면을 구현합니다.
- 6) 페어와 같이 추가적인 새로운 아이디어를 논의하고 정리합니다.
- 7) 추가적인 아이디어를 적용하여 구현하고 프로젝트를 완성합니다.
- 8) 산출물을 정리하여 GIT 에 반영합니다.

1.4 요구사항

본 프로젝트는 자동차 인포테인먼트 시스템을 중심으로 사용자 프로필 관리, 실시간 위치 기반 앱, 데이터 시각화 등 다양한 기능을 하나의 통합 장치에서 구현하는 것을 목표로 합니다. 최종적으로는 실제 차량 환경을 모사한 플랫폼 구축을 지향하며, 이번 로그 데이터 관리 프로젝트(PJT3)는 시스템 내에서 발생하는 대량의 데이터를 효율적으로 관리하는 기능에 초점을 맞춥니다. 차량 시스템의 원시 로그(Raw Log) 데이터를 수집 및 정제하여 구조화하고, 이를 웹 인터페이스상에서 실시간으로

시각화하는 기능을 구현합니다. 이를 통해 데이터의 전체 흐름(수집→정제→표시)을 경험하고, Firebase 기반의 서버리스 백엔드와 프론트엔드 연동 기술을 학습하는 것이 이번 요구사항의 핵심입니다.

■ 요구사항 예시 (참고용)

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현합니다. 단, 필수 기능은 반드시 구현해야 하며 수정할 수 없습니다.

기능적 요구사항				
번호	분류	요구사항명	요구사항 상세	우선순위
F201	기능	원시 로그 데이터 수집	시스템에서 발생하는 원시 로그 파일을 업로드하면 각 데이터를 수집 저장소에 자동으로 기록합니다.	필수
F202	기능	실시간 데이터 정제 처리	데이터가 저장소에 입력되는 즉시 사전에 정의된 정제 로직이 자동으로 실행되도록 구현합니다.	필수
F203	기능	구조화된 로그 데이터 저장	정제된 결과를 분석 및 조회가 용이하도록 일정한 규칙을 가진 구조화된 형태로 전용 저장소에 저장합니다.	필수
F204	화면	로그 실시간 시각화	데이터베이스의 변화를 감지하여 웹 화면에 별도의 새로그침 없이도 최신 로그가 즉시 표시되도록 구현합니다.	필수
F205	기능	데이터 상세 필터링 및 검색	사용자가 원하는 기간, 로그 등급, 특정 키워드 등을 조합하여 필요한 데이터를 빠르게 조회할 수 있도록 합니다	필수
F206	기능	로그 등급별 시각적 차별화	INFO, WARNING, ERROR 등 로그의 중요도 레벨에 따라 색상을 다르게 적용하여 가독성을 높입니다.	필수
F207	기능	사용자 맞춤형 로그 분석	특정 사용자 ID를 기준으로 데이터를 선별하여 개인별 시스템 이용 이력을	심화

			정밀하게 조회할 수 있는 기능을 추가합니다.	
--	--	--	-----------------------------	--

1. 기본 기능 (필수)

1) 시스템 원시 로그 업로드 기능을 구현합니다.

- ① 요구사항 번호 : F201
- ② 사용자가 로그 파일을 업로드하면 데이터를 행 단위로 분리하여 raw_telematics_logs 컬렉션에 저장합니다.
 - 대량의 데이터를 누락 없이 기록하기 위해 NoSQL 기반 저장 구조를 활용하며, 실시간 데이터 처리를 위한 기초 데이터셋을 형성합니다.

2) 서버리스 기반 자동 로그 정제 기능을 구현합니다.

- ① 요구사항 번호 : F202
- ② Firebase Functions 의 onDocumentCreated 트리거를 활용하여 데이터 생성 시 정제 로직이 자동 실행되도록 구현합니다.

3) 구조화된 정제 로그 저장 기능을 구현합니다.

- ① 요구사항 번호 : F203
- ② 정제 처리가 완료된 데이터를 telematics_logs 컬렉션에 규격화된 형태로 저장합니다.
 - 분석과 조회가 용이하도록 시간, 사용자 ID, 로그 레벨 등이 구분된 구조로 데이터를 보관하여 데이터 파이프라인을 완성합니다.

4) 로그 데이터 실시간 시각화 화면을 구현합니다.

- ① 요구사항 번호 : F204
- ② Firestore 의 실시간 구독(onSnapshot) 기능을 활용하여 화면 새로고침 없이 로그를 자동으로 출력합니다.

- 기존 인포테인먼트 메인 화면에서 로그 관리 화면으로 자연스럽게 진입하도록 제어하며, 최신 데이터가 상단에 노출되는 인터랙션을 제공합니다.

5) 실시간 로그 검색 및 필터링 기능을 구현합니다.

- ① 요구사항 번호 : F205
- ② 날짜 범위, 로그 레벨, 메시지 키워드별로 데이터를 필터링할 수 있는 검색 시스템을 구축합니다.
 - 대량의 로그 중 사용자가 원하는 특정 정보만 빠르게 선별하여 확인할 수 있도록 동적인 데이터 조회 환경을 제공합니다.

6) 로그 등급별 시각적 차별화 기능을 구현합니다.

- ① 요구사항 번호 : F206
- ② INFO, WARNING, ERROR 등 로그 등급에 따라 텍스트나 배경 색상을 다르게 적용합니다.
 - 관리자가 시스템의 이상 징후나 긴급 상황을 직관적으로 인지할 수 있도록 시각적 피드백을 강화합니다.

2. 심화 기능 (선택)

클라우드 서비스와 서버리스 환경을 기반으로 데이터의 수집부터 정제, 시각화에 이르는 전체 흐름을 직접 제어하여 시스템을 구축합니다. 이는 제작자가 백엔드와 프론트엔드 간의 유기적인 연동 원리를 깊이 있게 이해하고 있음을 보여주며, 정형화된 틀에서 벗어나 프로젝트의 목적에 최적화된 맞춤형 관리 시스템을 구현하는 데 그 목적이 있습니다.

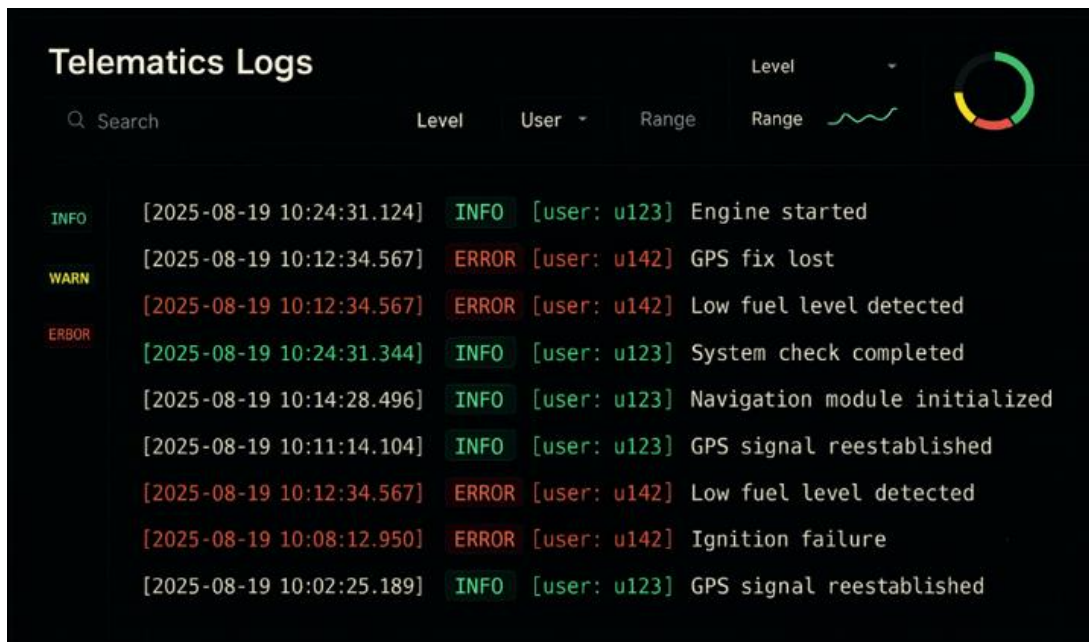
1) 사용자 ID 기반 상세 필터링 기능을 구현합니다.

- ① 요구사항 번호 : F207

- ② 특정 사용자 ID(log.user)를 기준으로 데이터를 선별하여 개별 활동 내역을 추적하는 기능을 추가합니다.
- 전체 로그 중 특정 사용자의 데이터만 빠르게 조회하여 문제 발생 원인을 정밀하게 분석하도록 지원합니다.

1.5 참고자료

- 프로젝트 구현에 필요한 데이터베이스 설계 및 쿼리 활용을 위한 Firebase Firestore 공식 문서입니다.
<https://firebase.google.com/docs/firestore>
- 원시 로그의 자동 정제 파이프라인 및 서버리스 로직 구현을 위한 Cloud Functions 공식 문서입니다.
<https://firebase.google.com/docs/functions>
- 정제된 로그 리스트를 시간순으로 나열하고 검색 필터를 배치하는 태블릿 기반 데이터 인터페이스 레이아웃 참고 이미지입니다.



<그림 3> 참고용 차량용 로그 데이터 관리 및 시각화 시스템

1.6 프로젝트 결과

본 프로젝트의 최종 결과물은 원활한 검수와 관리를 위해 지정된 양식에 따라

작성되어야 하며, 모든 산출물은 GitLab 저장소에 업로드하여 제출을 완료합니다. 특히 파일명 규칙은 데이터 관리의 일관성을 위해 반드시 준수해야 합니다.

■ 산출물과 제출

- 1) 프로젝트 수행으로 산출된 개선된 요구사항과 실행 화면 캡처본을 포함하여 최종 제출물을 작성합니다.
- 2) 최종 결과물은 README.md 파일이나 워드, 파워포인트 문서를 활용하여 작성하며, 프로젝트 소스 코드와 함께 압축하여 제출합니다.
- 3) 최종 제출 시 압축 파일의 명칭은 아래의 형식을 엄격히 준수하여 업로드합니다.

◆ 압축 파일명 : 임베디드_웹_관통 PJT2_지역_반_성명 1_성명 2.zip

- 以上